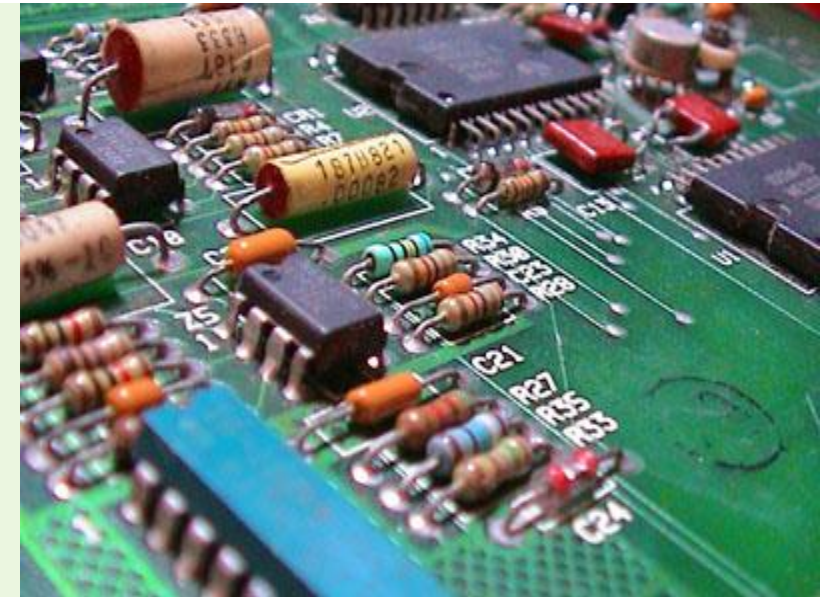




Embedded Systems and Robotics

ICE 372

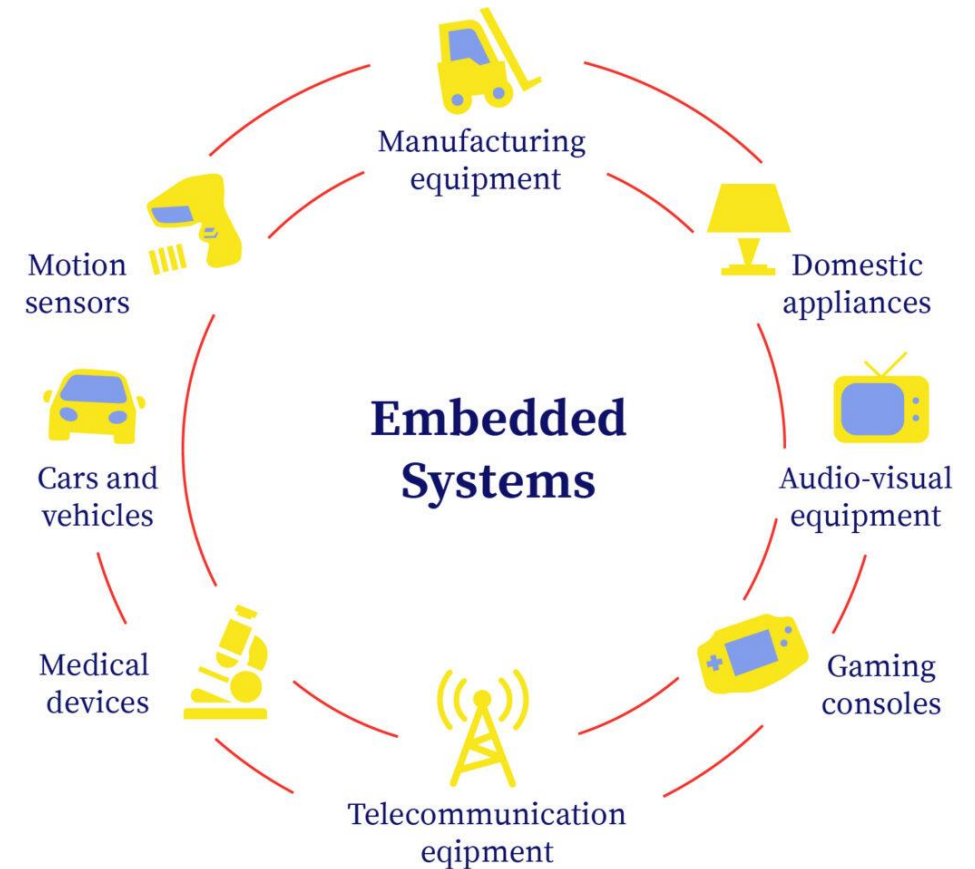
Lecture 1: Introduction



Dr. Emad Badry

What is an Embedded System?

- ❑ An embedded system is an electronic/electro-mechanical system designed to perform a specific function and is a combination of both hardware and firmware (software).
- ❑ Every embedded system is unique, and the hardware as well as the firmware is highly specialized to the application domain.
- ❑ Embedded systems are becoming an inevitable part of any product or equipment in all fields including household appliances, telecommunications, medical equipment, industrial control, consumer products, etc.



Classification of Embedded Systems

It is possible to have a multitude of classifications for embedded systems, based on different criteria. Some of the criteria used in the classification of embedded systems are as follows:

- (1) Based on generation
- (2) Complexity and performance requirements

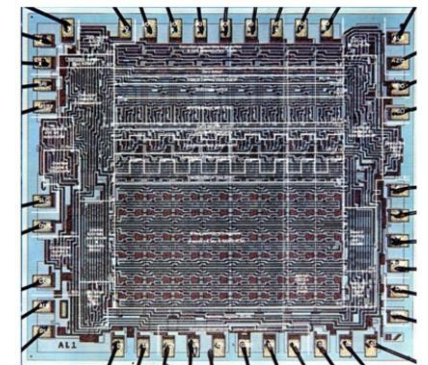
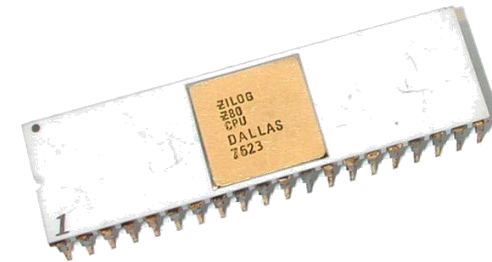
Classification Based on Generation

First Generation

The early embedded systems were built around **8-bit microprocessors** like 8085 and Z80, and **4-bit microcontrollers**. Simple in hardware circuits with firmware developed in **Assembly code**. Digital telephone keypads, stepper motor control units etc. are examples of this.

Second Generation

These are embedded systems built around **16-bit microprocessors** and **8 or 16 bit microcontrollers**, following the first generation embedded systems. The instruction set for the second generation processors/controllers were much more complex and powerful than the first generation processors/ controllers. Some of the second generation embedded systems contained embedded operating systems for their operation. Data Acquisition Systems, SCADA systems, etc. are examples of second generation embedded systems.



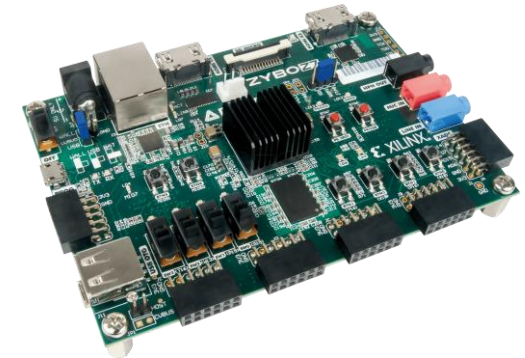
8-bit computer processor slice. Design commenced October 1968. Final working devices March 1969

Third Generation

With advances in processor technology, embedded system developers started making use of powerful **32-bit processors** and **16-bit microcontrollers** for their design. A new concept of application and domain specific processors/controllers like Digital Signal Processors (DSP) and Application Specific Integrated Circuits (ASICs) came into the picture. **The instruction set of processors became more complex and powerful** and the concept of instruction pipelining also evolved. The processor market was flooded with different types of processors from different vendors. Processors like Intel Pentium, Motorola 68K, etc. gained attention in high performance embedded requirements. Dedicated embedded real time and general purpose operating systems entered into the embedded market. Embedded systems spread its ground to areas like robotics, media, industrial process control, networking, etc.

Fourth Generation

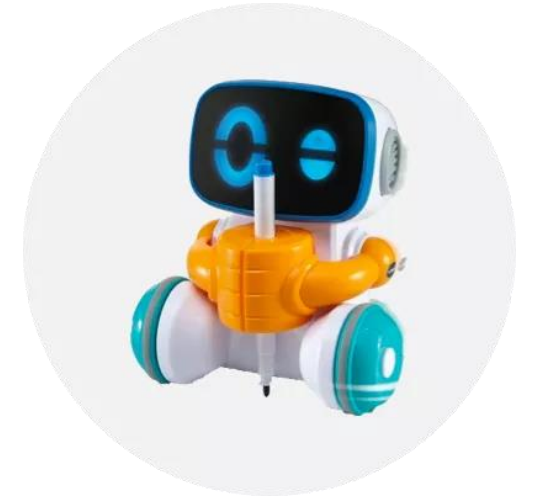
The advent of System on Chips (SoC), reconfigurable processors and multicore processors are bringing high performance, tight integration and miniaturization into the embedded device market. The SoC technique implements a total system on a chip by integrating different functionalities with a processor core on an integrated circuit. The fourth generation embedded systems are making use of high-performance real time embedded operating systems for their functioning. Smart phone devices, mobile internet devices (MIDs), etc. are examples of fourth generation embedded systems.



Classification Based on Complexity and Performance

Small-Scale Embedded Systems

Embedded systems which are simple in application needs and where the performance requirements are not time critical fall under this category. **An electronic toy is a typical example of a small-scale embedded system.** Small-scale embedded systems are usually built around low performance and low cost 8 or 16 bit microprocessors/microcontrollers. A small-scale embedded system may or may not contain an operating system for its functioning.



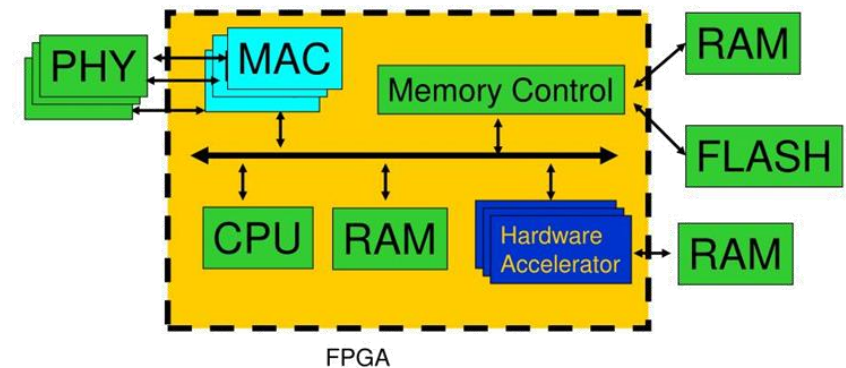
Medium-Scale Embedded Systems

Embedded systems which are slightly complex in hardware and firmware (software) requirements fall under this category. Medium-scale embedded systems are usually built around medium performance, low cost 16 or 32-bit microprocessors/microcontrollers or digital signal processors. **They usually contain an embedded operating system** (either general purpose or real time operating system) for functioning.

Large-Scale Embedded Systems/Complex Systems

Embedded systems which involve highly complex hardware and firmware requirements fall under this category. They are employed in mission critical applications demanding high performance. Such systems are commonly built around high performance 32 or 64 bit RISC processors/controllers or Reconfigurable System on Chip (RSoC) or multi-core processors and programmable logic devices. They may contain multiple processors/controllers and co-units/hardware accelerators for offloading the processing requirements from the main processor of the system. Complex embedded systems usually contain a high performance Real Time Operating System (RTOS) for task scheduling, prioritization, and management.

Reconfigurable System-on-Chip



Major Applications of Embedded Systems

The application areas and the products in the embedded domain are countless. A few of the important domains and products are listed below:

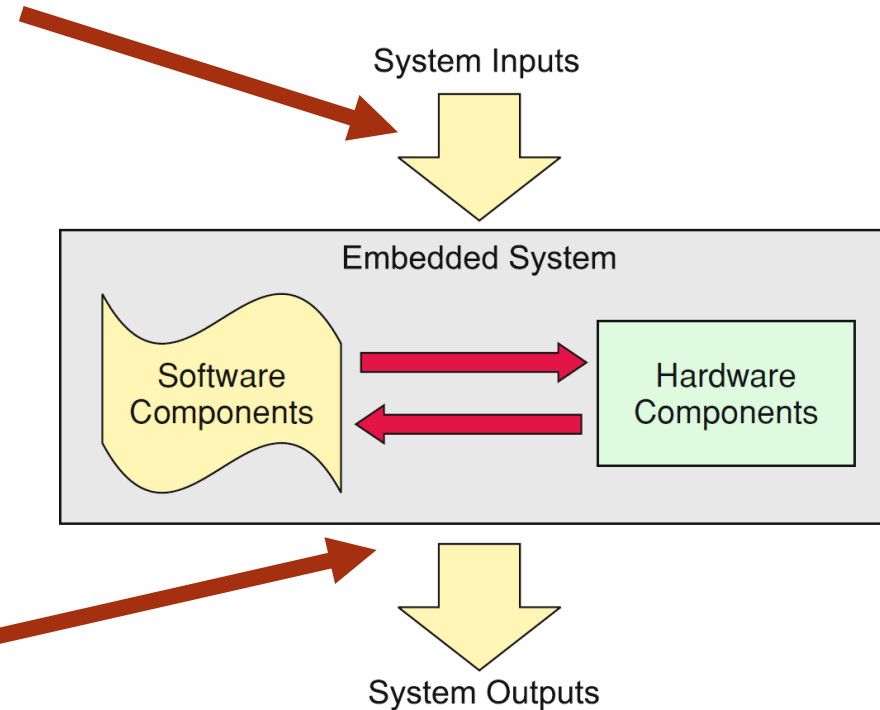
- (1) Consumer electronics
- (2) Household appliances
- (3) Home automation and security systems
- (4) Automotive industry
- (5) Telecom
- (6) Computer peripherals
- (7) Computer networking systems
- (8) Healthcare
- (9) Measurement & Instrumentation
- (10) Banking & Retail
- (11) Card Readers
- (12) Wearable Devices
- (13) Cloud Computing and Internet of Things (IOT)



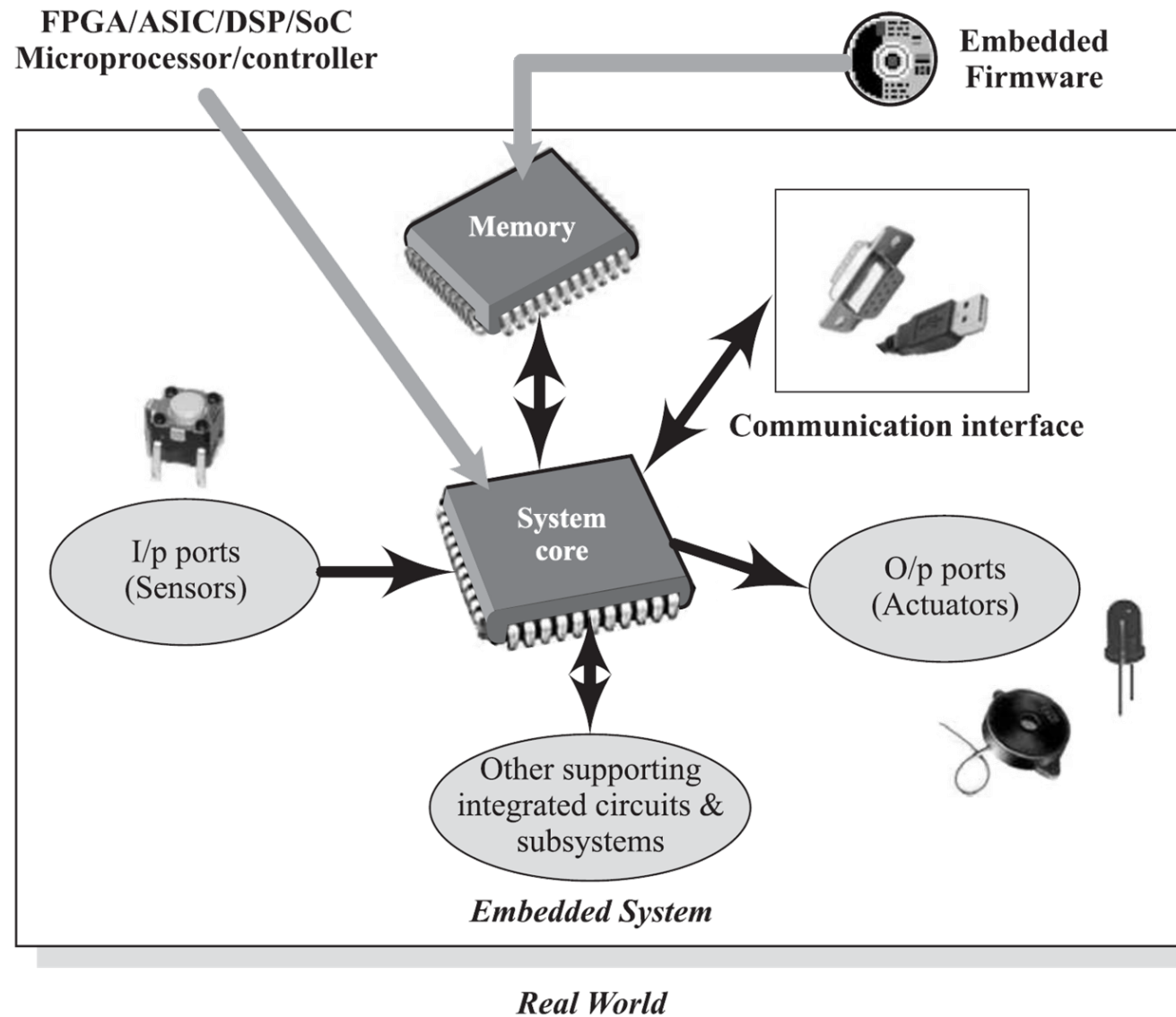
Structure of an Embedded System

Regardless of the function performed by an embedded system, the broadest view of its structure reveals two major, tightly coupled sets of components: **a set of hardware components that include a central processing unit, typically in the form of a microcontroller; and a series of software programs, typically included as firmware, that give functionality to the hardware.**

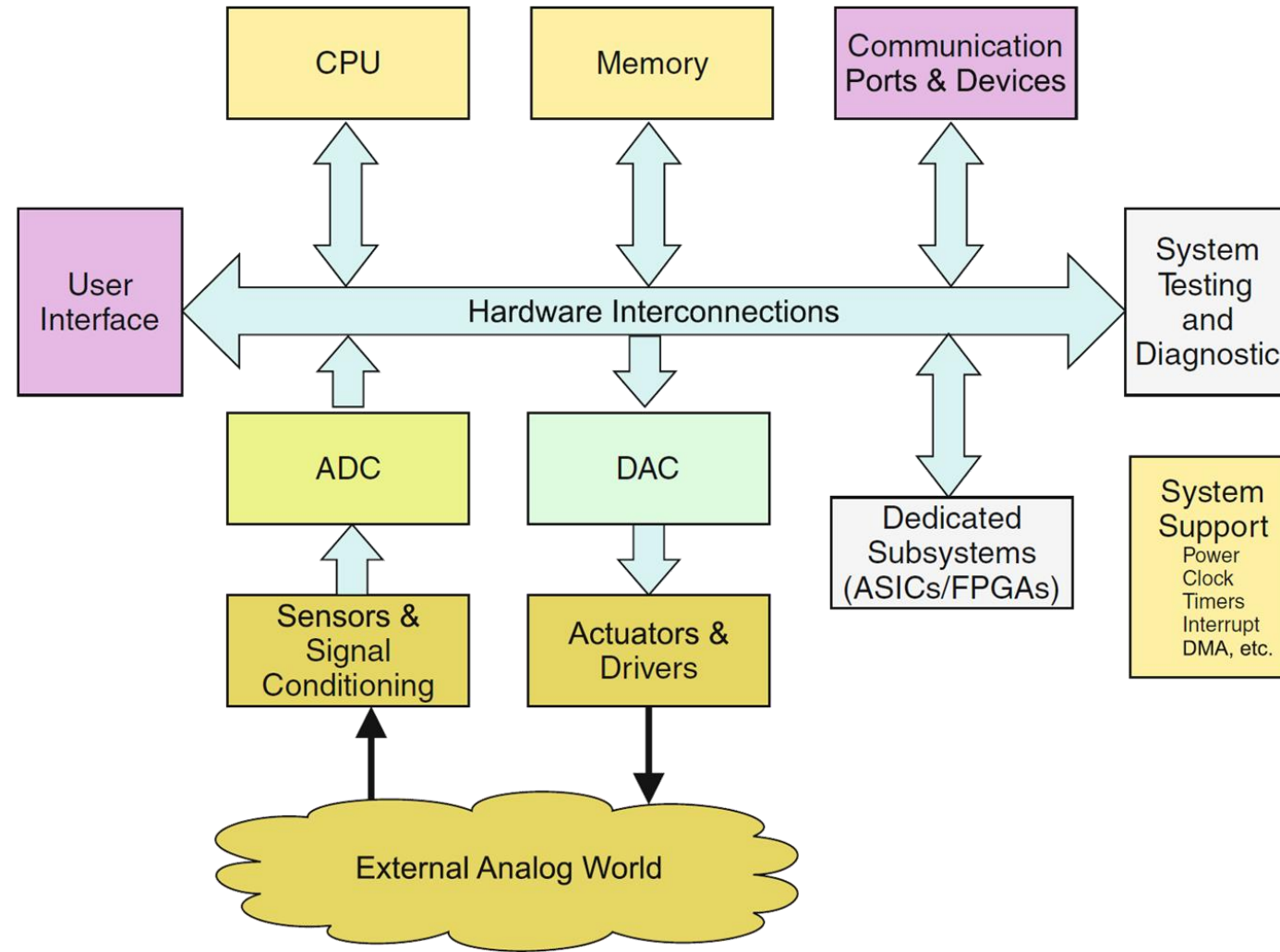
Process variables and parameters that arrive via sensors and input/output (I/O) ports.



Control actions on system actuators or processed information for users or other subsystems within the application



Hardware Components

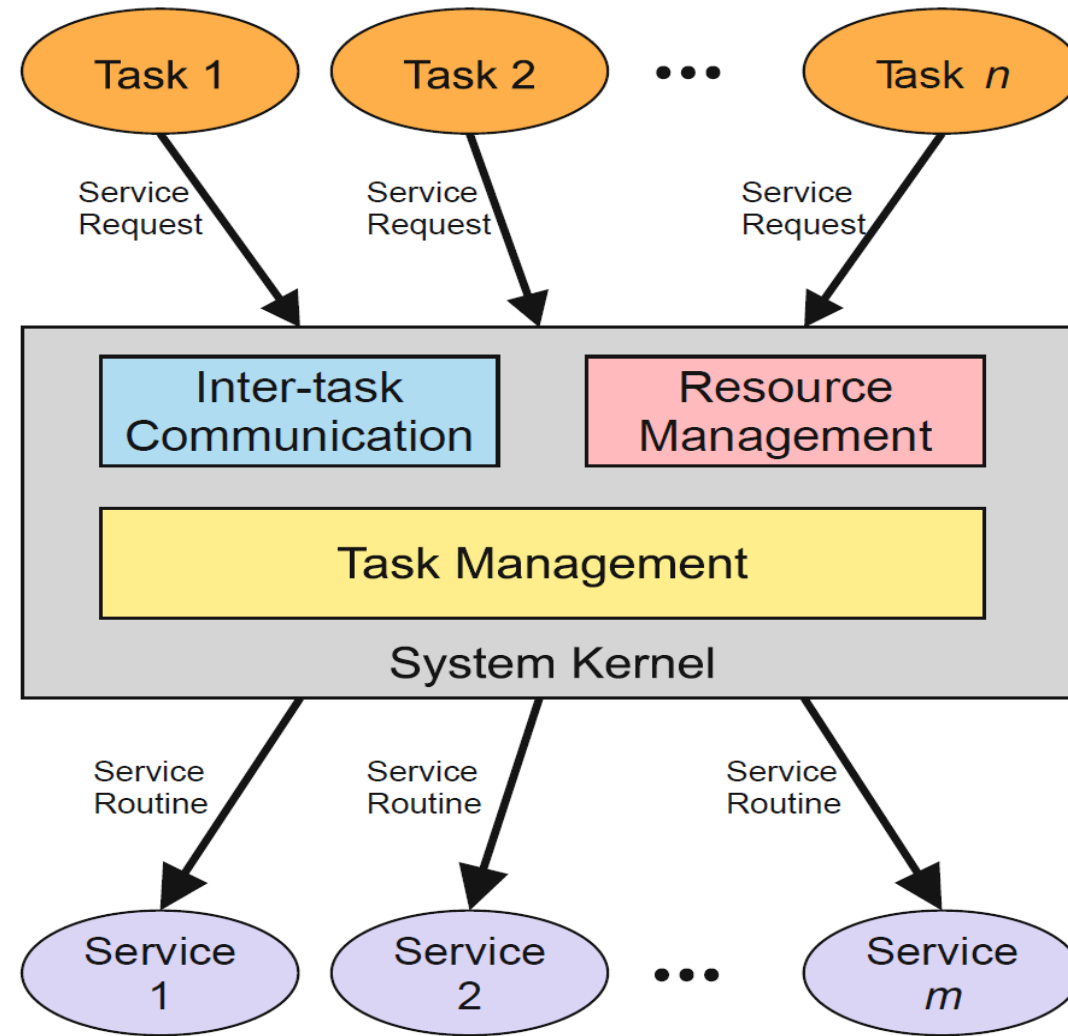


The hardware components of an embedded system include all the electronics necessary for the system to perform the function it was designed for. Therefore, the specific structure of a particular system could substantially differ from another, based on the application itself. Despite these dissimilarities, **three core hardware components are essential in an embedded system: The Central Processing Unit (CPU), the system memory, and a set of input-output ports.**

A number of other supporting and I/O devices needed for system functionality might be present, depending on the application. These include:

- ☐ Communication ports for serial and/or parallel information exchanges with other devices or systems.
- ☐ User interfaces to interact with humans.
- ☐ Sensors and electromechanical actuators to interact with the environment external to the system.
- ☐ Data converters (Analog-to-digital (ADC) and/or Digital-to-Analog (DAC)) to allow interaction with analog sensors and actuators.
- ☐ Diagnostics and redundant components to verify and provide for robust, reliable system operation.
- ☐ System support components to provide essential services that allow the system to operate.
- ☐ Other sub-systems to enable functionality, that might include Application Specific Integrated Circuits (ASIC), Field Programmable Gate Arrays (FPGA), and other dedicated units, according to the complexity of the application.

Software Components



The software components of an embedded system include all the programs necessary to give functionality to the system hardware. These programs, frequently referred to as the system firmware, are stored in some sort of non-volatile memory.

The major components identified in a system software include:

System Tasks

The application software in embedded systems is divided into a set of smaller programs called Tasks. **Each task handles a distinct action in the system and requires the use of specific System Resources.** Tasks submit service requests to the kernel in order to perform their designated actions.

System Kernel

The software component that handles the system resources in an embedded application is called the Kernel. System resources are all those components needed to serve tasks. These include memory, I/O devices, the CPU itself, and other hardware components. The kernel receives service requests from tasks, and schedules them according to the priorities dictated by the task manager. When multiple tasks contend for a common resource, a portion of the kernel establishes the resource management policy of the system. The kernel provides a framework that enables a reliable inter-task communication to exchange information and to coordinate collaborative operation.

Services.

Tasks are served through Service Routines. A service routine is a piece of code that gives functionality to a system resource. In some systems, **they are referred to as device drivers.** Services can be activated by polling or as interrupt service routines (ISR), depending on the system architecture.

Microprocessor vs Microcontroller

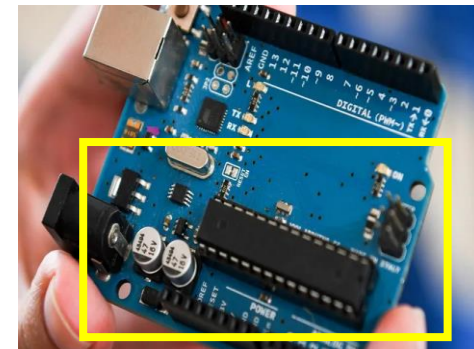
Microprocessor	Microcontroller
A silicon chip representing a central processing unit (CPU), which is capable of performing arithmetic as well as logical operations according to a predefined set of instructions.	Microcontroller is a highly integrated chip that contains a CPU, RAM, special and general-purpose register arrays, on chip ROM/FLASH memory for program storage, timer and interrupt control units and dedicated I/O ports.
It is a dependent unit. It requires the combination of other chips like timers, program and data memory chips, interrupt controllers, etc. for functioning.	It is a self-contained unit, and it doesn't require external interrupt controller, timer, UART, etc. for its functioning.
Doesn't contain a built in I/O port. The I/O port functionality needs to be implemented with the help of external programmable peripheral interface chips.	Most of the processors contain multiple built-in I/O ports which can be operated as a single 8 or 16- or 32-bit port or as individual port pins.
Targeted for high end market where performance is important.	Targeted for embedded market where performance is not so critical.
Limited power saving options compared to microcontrollers.	Includes lot of power saving features.



CPU



Microcontroller



Board with Microcontroller

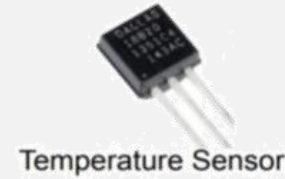
RISC vs. CISC Processors/Controllers

- ❑ The term *RISC* stands for Reduced Instruction Set Computing. As the name implies, all RISC processors/controllers possess lesser number of instructions, typically in the range of 30 to 40.
- ❑ *CISC* stands for Complex Instruction Set Computing. From the definition itself it is clear that the instruction set is complex and instructions are high in number.
- ❑ From a programmers point of view RISC processors are comfortable since s/he needs to learn only a few instructions, whereas for a CISC processor s/he needs to learn more number of instructions and should understand the context of usage of each instruction (This scenario is explained on the basis of a programmer following Assembly Language coding. For a programmer following C coding it doesn't matter since the cross-compiler is responsible for the conversion of the high level language instructions to machine dependent code).

SENSORS AND ACTUATORS

Sensors

A sensor is a transducer device that converts energy from one form to another for any measurement or control purpose.



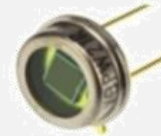
Temperature Sensor



Humidity Sensor



Proximity Sensor



Light Sensor



Metal Sensor



Color Sensor

Actuators

Actuator is a form of transducer device (mechanical or electrical) which converts signals to corresponding physical action (motion). Actuator acts as an output device.



DC Motor



DC Gear Motor



RC Servo motor



Stepper motor



BLDC Motor



Smart Servo motors



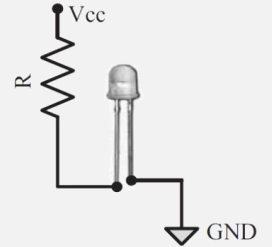
Harmonic drives



Linear electric actuator

The I/O Subsystem

- ❑ The I/O subsystem of the embedded system facilitates the interaction of the embedded system with the external world. The sensors may not be directly interfaced to the input ports, instead they may be interfaced through signal conditioning and translating systems like ADC, optocouplers, etc.

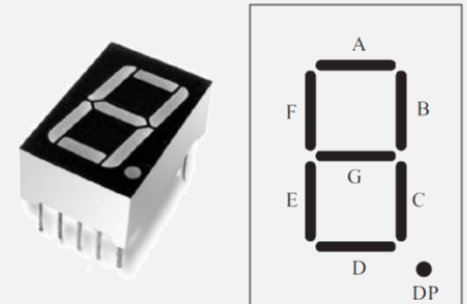


Light Emitting Diode (LED)

- ❑ Light Emitting Diode (LED) is an important output device for visual indication in any embedded system. LED can be used as an indicator for the status of various signals or situations. Typical examples are indicating the presence of power conditions like 'Device ON', 'Battery low' or 'Charging of battery' for a battery-operated handheld embedded devices.

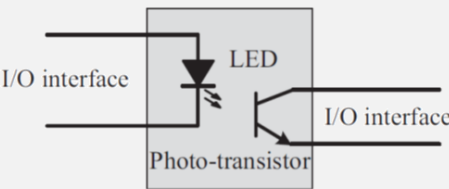
7-Segment LED Display

The 7-segment LED display is an output device for displaying alpha numeric characters. It contains 8 light-emitting diode (LED) segments arranged in a special form. Out of the 8 LED segments, 7 are used for displaying alpha numeric characters and 1 is used for representing 'decimal point' in decimal number display.

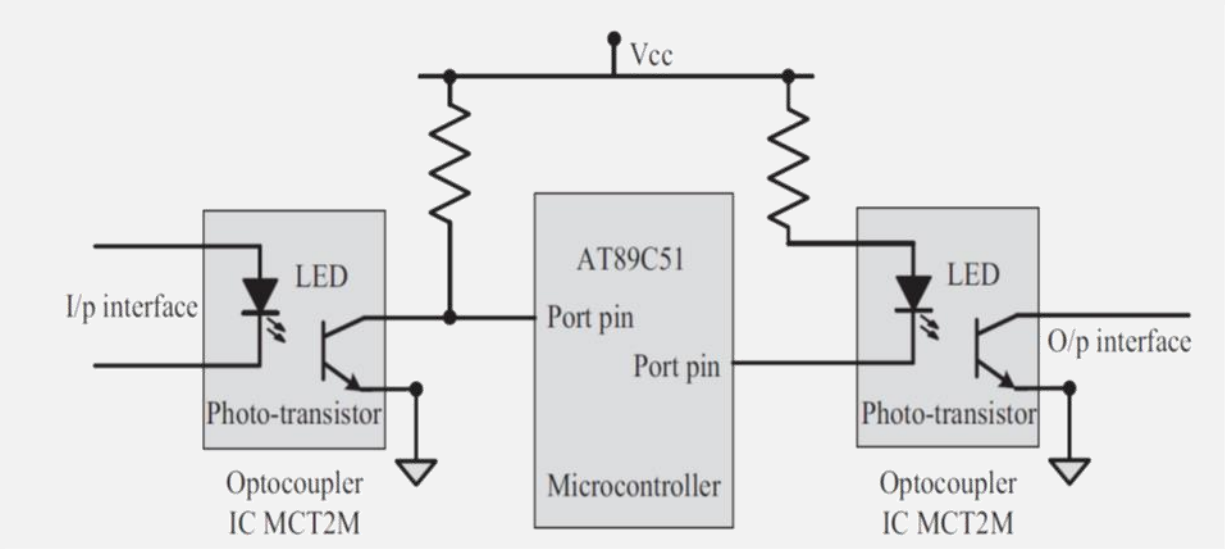


Optocoupler

Optocoupler is a solid state device to isolate two parts of a circuit. Optocoupler combines an LED and a photo-transistor in a single housing (package).

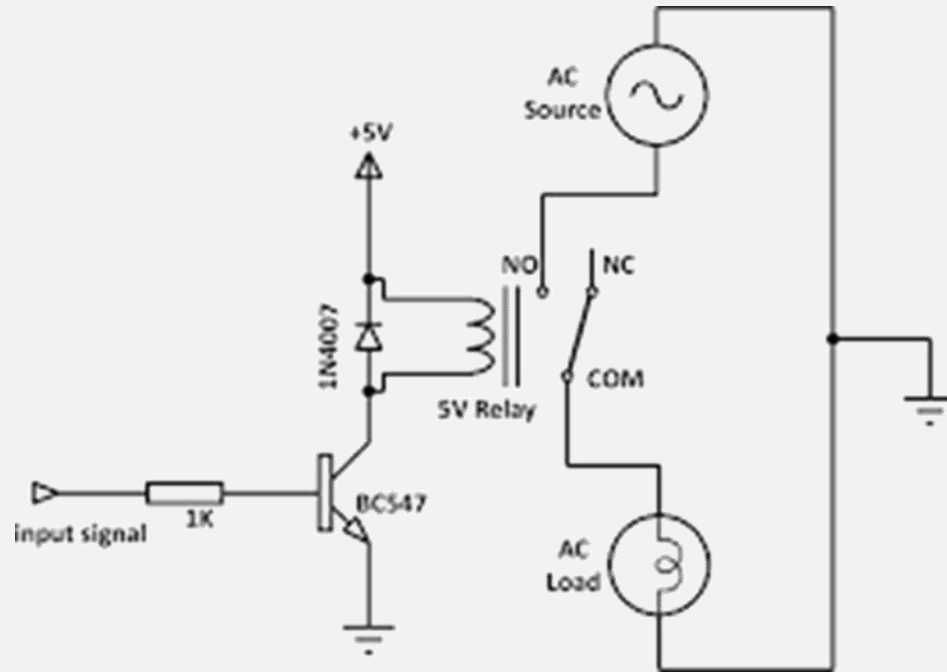


In electronic circuits, **an optocoupler is used for suppressing interference in data communication**, circuit isolation, high voltage separation, simultaneous separation and signal intensification, etc. Optocouplers can be used in either input circuits or in output circuits.



Relay

Relay is an electro-mechanical device. In embedded application, the 'Relay' unit acts as dynamic path selectors for signals and power. The 'Relay' unit contains a relay coil made up of insulated wire on a metal core and a metal armature with one or more contacts. 'Relay' works on electromagnetic principle. When a voltage is applied to the relay coil, current flows through the coil, which in turn generates a magnetic field. The magnetic field attracts the armature core and moves the contact point.



Push Button Switch

In embedded application push button is generally used as reset and start switch and pulse generator. The Push button is normally connected to the port pin of the host processor/controller. Depending on the way in which the push button interfaced to the controller, it can generate either a 'HIGH' pulse or a 'LOW' pulse.

Keyboard

Keyboard is an input device for user interfacing. If the number of keys required is very limited, push button switches can be used and they can be directly interfaced to the port pins for reading. Matrix keyboard is an optimum solution for handling large key requirements. It greatly reduces the number of interface connections. For example, for interfacing 16 keys, in the direct interfacing technique 16 port pins are required, whereas in the matrix keyboard only 8 lines are required.

